AI-Powered Customer Churn Prediction in the Telecom Industry

Mazharul Islam (22201120) Afran Anas (22299288) Mst. Khadija (22301311)
BRAC University

Abstract

Using the Telco Customer Churn data set, which has 7,043 customers, 19 predictors and a binary target, we trained three supervised classifiers on a stratified 70/30 split: logistic regression, a depth-limited decision tree and a neural network with two hidden layers. We also applied k-means clustering with $k = 3$ without using the churn label. On the 2,113 test customers, the classifiers reached accuracies between 79.1% and 80.4% and ROC AUC values between 0.83 and 0.85. For the churn class, precision ranged from 0.62 to 0.68 and recall from 0.50 to 0.53. The three clusters had churn rates of 47.7%, 15.6% and 15.2%.

1 Introduction

In this project we use account and service attributes of telecom customers to predict whether a customer will churn, and we compare three supervised classifiers for this task. We also cluster the customers with k-means, without using the churn label, and then look at the churn rate of each cluster.

2 Data

2.1 Overview

We use the Telco Customer Churn data set, which has 7,043 rows, one per customer, and 21 columns. After dropping the identifier `customerID`, we are left with 20 columns: 19 predictors and the target `Churn`, which takes the values "Yes" and "No". The task is therefore a binary classification problem.

2.2 Feature types

Three predictors are numeric: `tenure`, `MonthlyCharges` and `TotalCharges`. The other 16 predictors are categorical: `gender`, `SeniorCitizen`, `Partner`, `Dependents`, `PhoneService`, `MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`, `Contract`, `PaperlessBilling` and `PaymentMethod`. `SeniorCitizen` is stored as 0 or 1 in the file, and we convert it to a string so that it is treated as a categorical variable. `TotalCharges` is read in as text and is converted to a number during preprocessing (Section 3).

2.3 Class balance and distributions

The target is imbalanced: 73.5% of the customers did not churn and 26.5% did. Among the categorical predictors, 55.0% of the customers are on a month-to-month contract, 24.1% on a

two-year contract and 20.9% on a one-year contract. For internet service, 44.0% have fiber optic, 34.4% have DSL and 21.7% have no internet service. Electronic check is the most common payment method (33.6%), and 50.5% of the customers are male.

The skewness is 0.24 for `tenure`, -0.22 for `MonthlyCharges` and 0.96 for `TotalCharges`.

3 Preprocessing

Missing values: The check `df.isnull().sum()` reports no missing values, but `TotalCharges` is stored with type `object`. We convert it to a numeric type with `pd.to_numeric(errors="coerce")`, which turns entries that cannot be parsed into missing values, and we fill these with the median of the column. In our notebook this step is carried out on the full data frame, before the train/test split.

Splitting: We split the data 70/30 with `train_test_split(stratify=y, random_state=0)`. The test set contains 2,113 customers, of whom 1,552 did not churn and 561 did. Stratification keeps the class ratio the same in the training and test sets. We did not use resampling or class weights.

Encoding: `Contract` and `PaymentMethod` are one-hot encoded with `pd.get_dummies`, which gives three and four indicator columns respectively. All other categorical columns, including multi-level ones such as `InternetService` and `MultipleLines`, are integer-coded with `LabelEncoder`, which is fitted on the training set and then applied to the test set. The target is coded as No $= 0$ and Yes $= 1$.

Scaling: Only `TotalCharges` is rescaled, using a `MinMaxScaler` fitted on the training set. `tenure` and `MonthlyCharges` keep their original units, and we apply no other scaler or power transform.

4 Models

All models are scikit-learn implementations trained on the same encoded training matrix. Any parameter that is not mentioned below was left at its scikit-learn default.

Logistic regression: We set `max_iter=5000` and `random_state=42`.

Decision tree: We set `max_depth=5`, `min_samples_leaf=10` and `random_state=42`.

Neural network (MLP): The network has two hidden layers of 30 units each, with ReLU activation and the Adam solver. We set `learning_rate_init=0.001`, `max_iter=500` and `random_state=42`.

k-means: We use `n_init=10` and `random_state=42`, and fit the model on the encoded training feature matrix without the churn label. We compute the inertia for $k = 2, \ldots, 10$ and then use $k = 3$.

5 Results

5.1 Supervised models

Table 1 shows the confusion matrices on the test set, and Table 2 summarises the metrics.

Model	TN	FP	FN	TP
Logistic regression	1381	171	264	297
Decision tree	1372	180	262	299
Neural network	1417	135	280	281

Table 1: Test-set confusion matrices, with churn as the positive class. The values are read from the confusion-matrix figures in the notebook.

			Churn = Yes			Churn = No
Model	Accuracy	AUC	Precision	Recall	F1	Recall
Logistic regression	0.794	0.85	0.63	0.53	0.58	0.89
Decision tree	0.791	0.83	0.62	0.53	0.57	0.88
Neural network	0.804	0.85	0.68	0.50	0.58	0.91

Table 2: Test-set metrics taken from the accuracy printout and the classification reports in the notebook. The AUC values are read from the legend of Figure 1.

Logistic regression reached an accuracy of 79.4%, the decision tree 79.1% and the neural network 80.4%. In the test set, 73.5% of the customers (1,552 of 2,113) did not churn. Logistic regression and the neural network both have an AUC of 0.85, while the decision tree has 0.83 (Figure 1). The neural network has the highest precision for the churn class (0.68) and the highest recall for the non-churn class (0.91), but it also has the lowest recall for the churn class (0.50). Logistic regression and the decision tree both have a churn recall of 0.53, detecting 297 and 299 of the 561 churners respectively.

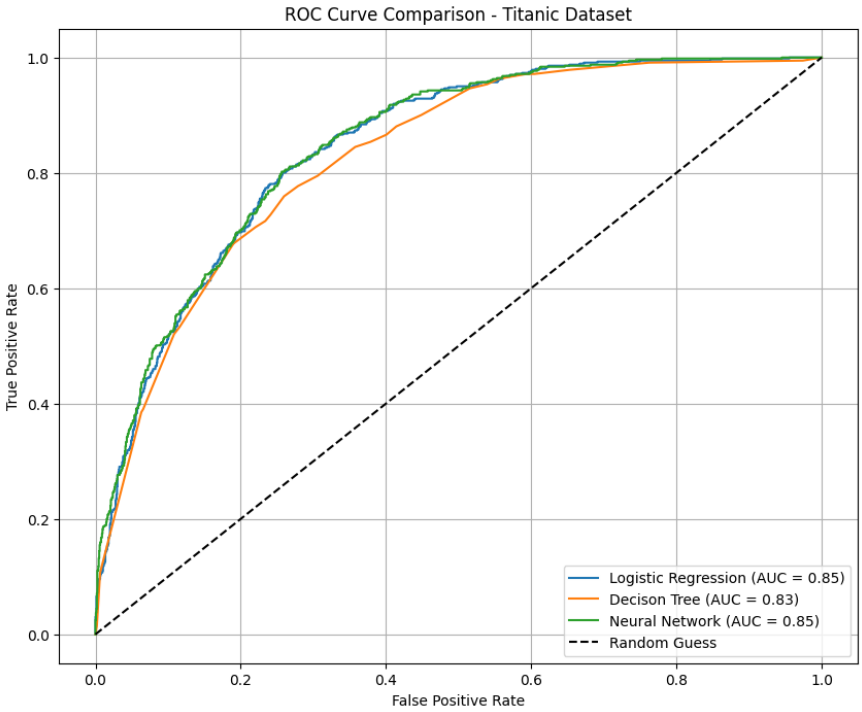


Figure 1: ROC curves on the test set, as saved in the notebook. The curves for logistic regression and the neural network are almost identical (AUC 0.85), and the decision tree has an AUC of 0.83. The title printed on the original plot reads "Titanic Dataset".

5.2 Customer segmentation with k-means

Figure 2 shows the inertia for $k = 2, \ldots, 10$, and in the notebook we then fit $k = 3$.

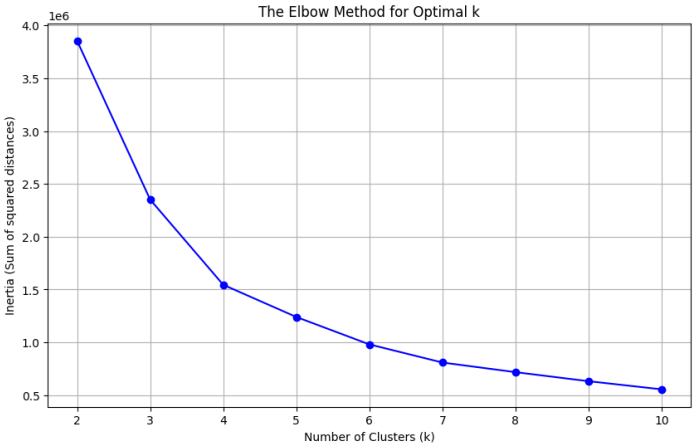


Figure 2: k-means inertia for $k = 2, \ldots, 10$ on the training set.

Table 3 gives the mean tenure, monthly charge and scaled total charge of each cluster, together with its churn rate, which is also shown in Figure 3. The churn label was not used to fit the clusters; we only used it afterwards to compute each cluster's churn rate.

Cluster	Tenure (months)	Monthly charge	TotalCharges (scaled)	Churn rate
0	13.53	77.07	0.12	47.7%
1	57.98	89.75	0.60	15.6%
2	27.56	28.43	0.09	15.2%

Table 3: Cluster means and churn rates on the training set. `TotalCharges` is the min–max scaled value.

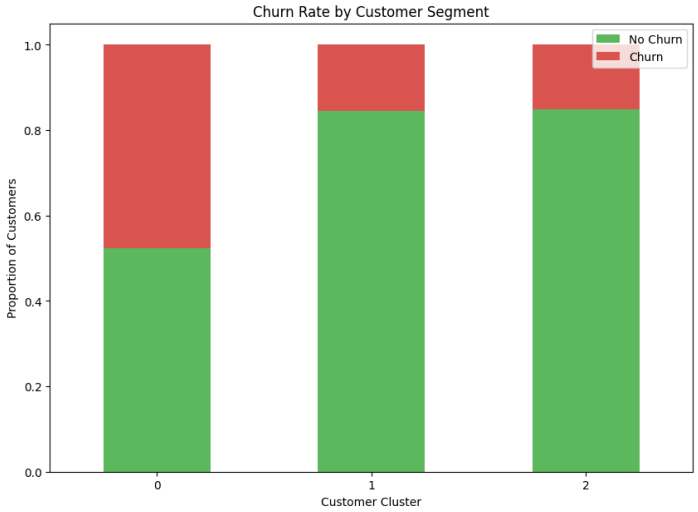


Figure 3: Proportion of non-churners and churners in each cluster.

Cluster 0 has the shortest mean tenure and the highest churn rate (47.7%). Cluster 1 has the longest mean tenure and the highest mean monthly charge, and its churn rate is 15.6%. Cluster 2 has the lowest mean monthly charge and a mean tenure between those of the other two clusters, and its churn rate is 15.2%.

6 Conclusion

On the 2,113-customer test set, logistic regression, the decision tree and the neural network reached accuracies of 79.4%, 79.1% and 80.4%, with AUC values of 0.85, 0.83 and 0.85. The neural network had the highest accuracy and the highest precision for churners, logistic regression and the decision tree had the highest churn recall (0.53), and logistic regression tied with the neural network on AUC. Each model detected between 281 and 299 of the 561 churners in the test set. The k-means clustering with $k = 3$ produced one cluster with a mean tenure of 13.5 months and a churn rate of 47.7%, and two clusters with churn rates of 15.6% and 15.2%.